



Introduction to React

Components, Props, State & Event Handling with TypeScript

Chapter Overview

- 01** What Is React?
- 02** Components and JSX
- 03** Props
- 04** State and Events
- 05** Conditional Rendering and Lists
- 06** Component Composition



What Is React?

Section 1



Why React?

The Problem

- Manual DOM updates are complex and error-prone
- Keeping UI in sync with changing data requires careful, repetitive code

React's Solution

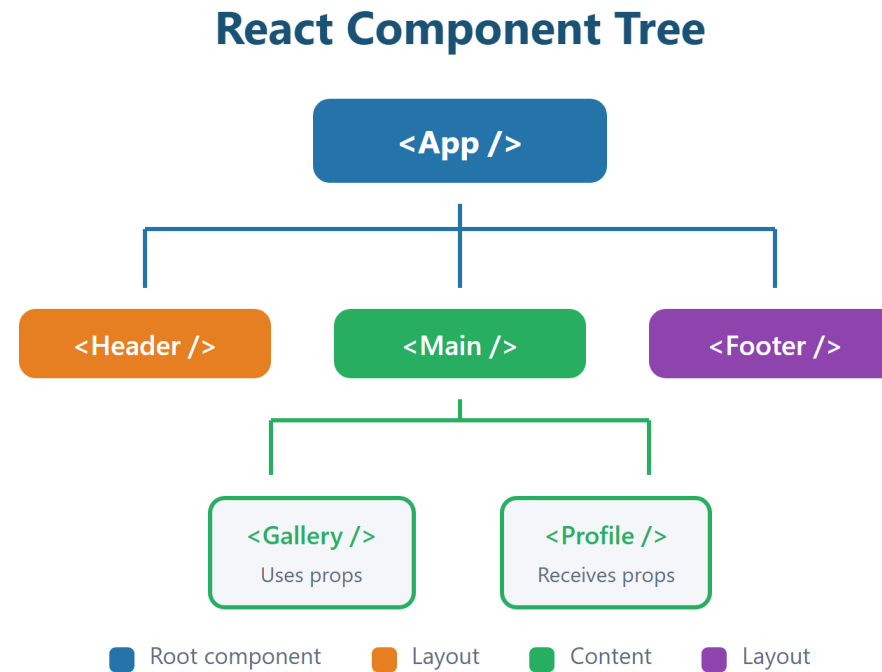
- Build UIs from reusable components — like LEGO blocks
- React updates only what changed — no full page reloads
- Declarative: describe WHAT the UI should look like, React figures out HOW

Why It Won

- Used by Meta, Netflix, Airbnb, Uber, and most modern web applications
- Massive ecosystem: React Router, Redux, Next.js, React Native



React Component Tree



Components compose into a tree — data flows down via props from parent to child

How React Works

The Virtual DOM

- React keeps a lightweight copy of the DOM in memory
- When state changes, React compares old and new virtual DOM (diffing)
- Only the differences are applied to the real DOM (reconciliation)

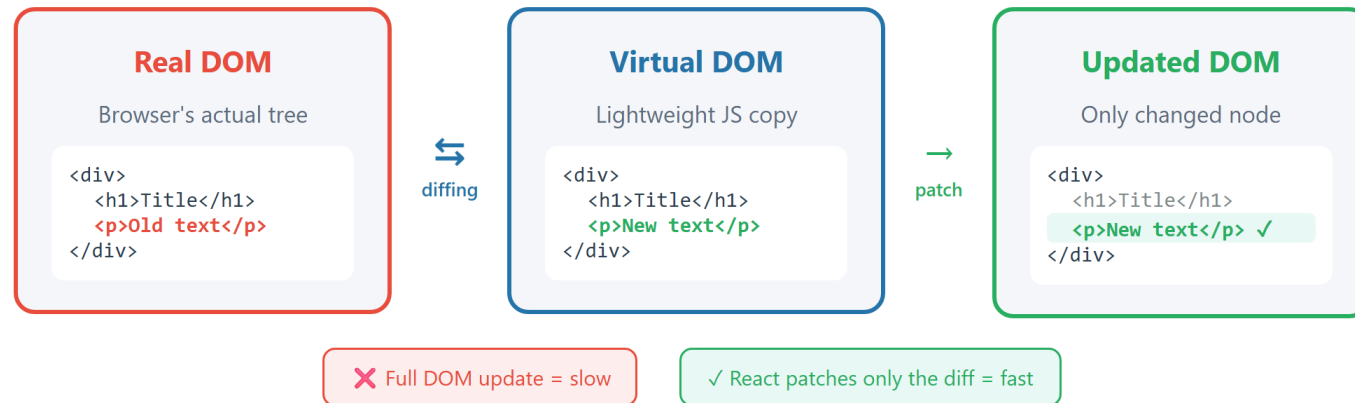
One-Way Data Flow

- Data flows down from parent to child via props
- Events flow up from child to parent via callback functions
- This makes data flow predictable and easy to debug



The Virtual DOM

The Virtual DOM — How React Updates Efficiently



React schedules and reconciles updates via the Fiber architecture — compares Virtual DOM snapshots, calculates minimal changes, and batches DOM updates

React + TypeScript + Vite

Why TypeScript with React?

- Type-safe props — catch wrong prop types at compile time, not runtime
- Typed state — `useState<number>(0)` prevents accidental string assignment
- Autocomplete for every component's props, events, and hooks

Why Vite?

- Instant dev server startup — no waiting for Webpack builds
- Hot Module Replacement — changes appear in milliseconds
- Built-in TypeScript transpilation — type checking handled separately (e.g., tsc or IDE)



Create a React Project

```
# Scaffold with Vite
npm create vite@latest my-app -- --template react-ts

# Install dependencies
cd my-app
npm install

# Start the dev server
npm run dev
# → http://localhost:5173/

# Project structure:
#   src/main.tsx    – entry point
#   src/App.tsx     – root component
#   src/App.css     – styles
#   vite.config.ts  – Vite config
```



Components and JSX

Section 2



What Is a Component?

Definition

- A component is a function that returns JSX (HTML-like syntax)
- Components are the building blocks of every React application

Rules

- Component names must start with a capital letter (PascalCase)
- Must return a single root element (use `<div>` or `<>` fragment)
- Function components are the standard — class components are legacy

File Convention

- One component per file — `UserCard.tsx` exports function `UserCard`
- `.tsx` extension for files containing JSX, `.ts` for plain TypeScript



Your First Component

```
// Greeting.tsx
function Greeting() {
  return (
    <div>
      <h1>Welcome to React</h1>
      <p>This is a function component.</p>
    </div>
  );
}

export default Greeting;

// App.tsx – use the component
import Greeting from './Greeting';

function App() {
  return <Greeting />;
}
```



JSX Fundamentals

JSX = JavaScript + HTML

- JSX looks like HTML but compiles to JavaScript function calls
- You can embed any JavaScript expression inside curly braces { }

Key Differences from HTML

- className instead of class — because class is a reserved word in JS
- htmlFor instead of for — same reason
- Self-closing tags required: not
- Style accepts objects: style={{ color: 'red' }}

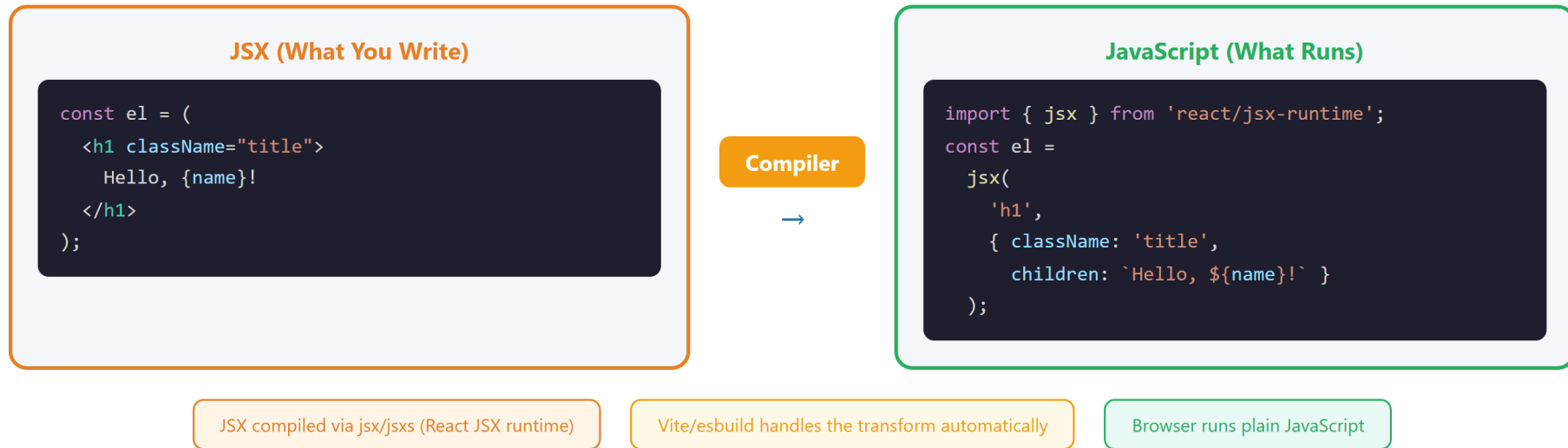
Expressions in JSX

- {variable}, {condition && <el/>}, {array.map(...)}



JSX Transform

JSX Transformation — From JSX to JavaScript



JSX Expressions

```
function UserCard() {
  const name = "Alice";
  const isAdmin = true;
  const skills = ['React', 'TypeScript', 'CSS'];

  return (
    <div className='card'>
      <h2>{name}</h2>
      {isAdmin && <span className='badge'>Admin</span>}
      <ul>
        {skills.map((skill) => (
          <li key={skill}>{skill}</li>
        ))}
      </ul>
    </div>
  );
}
```



Props

Section 3



What Are Props?

Props Are Arguments to Components

- Parent passes data to child via attributes: `<UserCard name='Alice' />`
- Child receives props as a typed object parameter

Props Are Read-Only

- A component must NEVER modify its own props
- Props flow one direction: parent → child (one-way data flow)

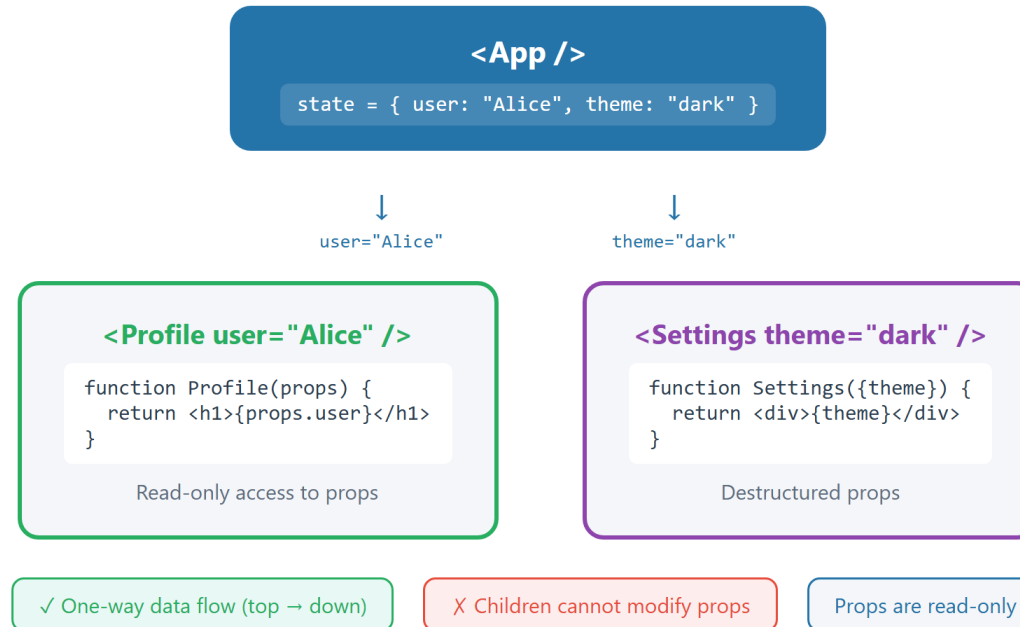
Typing Props with TypeScript

- Define an interface for props — TypeScript checks every usage
- Optional props use `?:` — the parent doesn't have to pass them



Props Data Flow

Props — One-Way Data Flow



Typed Props

```
// UserCard.tsx
interface UserCardProps {
  name: string;
  role: string;
  isActive?: boolean; // optional, defaults to undefined
}

function UserCard({ name, role, isActive = true }: UserCardProps) {
  return (
    <div className={`card ${isActive ? 'active' : 'inactive'}`}>
      <h2>{name}</h2>
      <p>{role}</p>
    </div>
  );
}

// Usage: <UserCard name='Alice' role='Developer' />
```

Children Props

The children Prop

- Content between opening and closing tags becomes props.children
- `<Card><h2>Title</h2></Card>` — the h2 is children

Typing Children

- children: `React.ReactNode` — accepts any renderable content
- This pattern creates reusable wrapper/layout components



Children Props in Practice

```
// Card.tsx – a reusable wrapper component
interface CardProps {
  title: string;
  children: React.ReactNode;
}

function Card({ title, children }: CardProps) {
  return (
    <div className='card'>
      <h2>{title}</h2>
      <div className='card-body'>{children}</div>
    </div>
  );
}

// Usage:
// <Card title='User Info'>
//   <p>Alice – Developer</p>
// </Card>
```



State and Events

Section 4



useState — Managing Component State

What Is State?

- State is data that changes over time — counts, form inputs, toggles
- When state changes, React re-renders the component automatically

useState Hook

- `const [value, setValue] = useState<Type>(initialValue)`
- Returns a pair: the current value and a function to update it
- Never mutate state directly — always use the setter function

Rules of Hooks

- Only call hooks at the top level — never inside loops or conditions
- Only call hooks from React function components (not regular functions)



useState Examples

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState<number>(0);

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

Event Handling in React

React Events

- `onClick`, `onChange`, `onSubmit` — camelCase event names on JSX elements
- Event handlers receive a React synthetic event (not pooled since React 17+)

Typing Events

- `React.MouseEvent<HTMLButtonElement>` for click handlers
- `React.ChangeEvent<HTMLInputElement>` for input changes
- `React.FormEvent<HTMLFormElement>` for form submissions

Inline vs Named Handlers

- Inline: `onClick={() => setCount(count + 1)}` — fine for simple logic
- Named: `const handleClick = () => {...}` — cleaner for complex logic



Typed Event Handlers

```
import { useState } from 'react';

function SearchForm() {
  const [query, setQuery] = useState<string>('');

  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    setQuery(e.target.value);
  };

  const handleSubmit = (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    console.log('Searching for:', query);
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={query} onChange={handleChange} />
      <button type='submit'>Search</button>
    </form>
  );
}
```



State with Objects and Arrays

Object State

- Use spread to create a new object: `setUser({...user, name: 'Bob'})`
- Never mutate: `user.name = 'Bob'` will NOT trigger a re-render

Array State

- Add: `setItems([...items, newItem])`
- Remove: `setItems(items.filter(i => i.id !== id))`
- Update: `setItems(items.map(i => i.id === id ? {...i, done: true} : i))`

Why Immutability?

- React compares old and new state by reference — mutation is invisible
- Always create a new object or array instead of modifying the existing one



State with Arrays

```
interface Todo {  
  id: number;  
  text: string;  
  done: boolean;  
}  
  
const [todos, setTodos] = useState<Todo[]>([]);  
  
// Add a todo  
const addTodo = (text: string) => {  
  setTodos([...todos, { id: Date.now(), text, done: false }]);  
};  
  
// Toggle a todo  
const toggleTodo = (id: number) => {  
  setTodos(todos.map(t =>  
    t.id === id ? { ...t, done: !t.done } : t  
  ));  
};  
  
// Remove a todo  
const removeTodo = (id: number) => {  
  setTodos(todos.filter(t => t.id !== id));  
};
```



Conditional Rendering and Lists

Section 5



Conditional Rendering

Three Patterns

- Logical AND: `{isAdmin && <AdminPanel />}` — render only if true
- Ternary: `{isLoggedIn ? <Dashboard /> : <Login />}` — either/or
- Early return: `if (!user) return <Loading />` — guard pattern

When to Use Each

- AND (&&) for optional elements — badges, warnings, extra details
- Ternary for toggling between two views
- Early return for loading/error states at the top of a component



Conditional Rendering Patterns

```
function UserProfile({ user }: { user: User | null }) {  
  // Early return for loading state  
  if (!user) return <p>Loading...</p>;  
  
  return (  
    <div>  
      <h2>{user.name}</h2>  
  
      {/* AND pattern – optional badge */}  
      {user.isAdmin && <span className='badge'>Admin</span>}  
  
      {/* Ternary – toggle between views */}  
      {user.isActive  
        ? <p className='active'>Active</p>  
        : <p className='inactive'>Inactive</p>  
      }  
    </div>  
  );  
}
```



Rendering Lists

Array.map() in JSX

- Use .map() to transform an array of data into an array of JSX elements
- `{items.map(item => <li key={item.id}>{item.name}</li>)}`

Keys Are Required

- Every element in a list must have a unique key prop
- Keys help React identify which items changed, added, or removed
- Use a unique ID from data — NEVER use array index as key

Filtering Lists

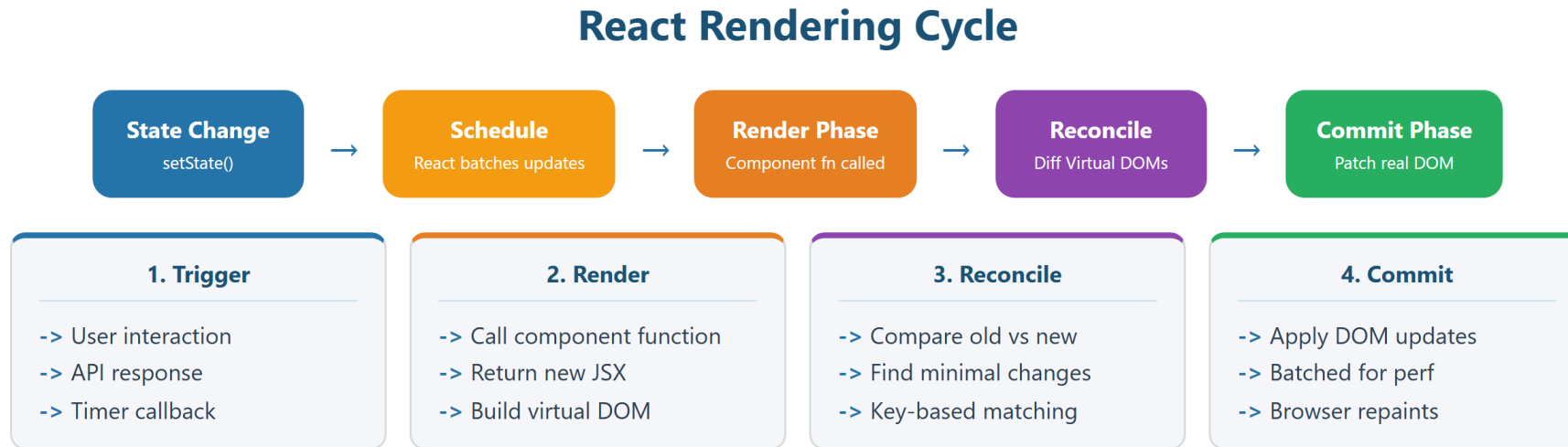
- Chain .filter() before .map(): `items.filter(...).map(...)`



Lists with Keys

```
interface Product {  
  id: number;  
  name: string;  
  price: number;  
  inStock: boolean;  
}  
  
function ProductList({ products }: { products: Product[] }) {  
  const available = products.filter(p => p.inStock);  
  
  return (  
    <ul>  
      {available.map(product => (  
        <li key={product.id}>  
          {product.name} - ${product.price.toFixed(2)}  
        </li>  
      ))}  
    </ul>  
  );  
}
```

React Rendering Cycle



React schedules and batches state updates, then commits changes in a single pass for optimal performance

Component Composition

Section 6



Building with Components

Component Hierarchy

- Break the UI into a tree of components — each does one thing
- App → Header + Main + Footer. Main → ProductList → ProductCard

When to Split

- If a section of JSX can be described with a noun, it's a component
- If you need to reuse UI in multiple places, extract a component
- If the component is getting too long (>50 lines), split it



React Project Structure

my-react-app/

```
|— public/
|   └─ favicon.ico
|— src/
|   └─ App.tsx
|   └─ App.css
|   └─ main.tsx
|   └─ components/
|       └─ Header.tsx
|       └─ Profile.tsx
|— index.html
|— vite.config.ts
|— tsconfig.json
|— eslint.config.js
|— package.json
|— node_modules/
└─ README.md
```

React Project Structure

index.html

Entry HTML at project root (not in public/). Vite injects the script tag. React mounts into `<div id="root">`

src/

Source code. `main.tsx` is the entry point. `App.tsx` is the root component.

components/

Reusable UI components. One component per `.tsx` file, PascalCase naming.

Config files

`vite.config.ts` — build config & plugins. `tsconfig.json` — TypeScript. `eslint.config.js` — ESLint flat config.

node_modules/

Installed packages (auto-generated). Never edit or commit to Git.

Created with `npm create vite@latest my-react-app -- --template react-ts`



Passing Callbacks to Children

Events Flow Up

- Parent passes a callback function as a prop
- Child calls the callback when something happens (click, submit)
- This is how child components communicate with parents

Common Pattern

- Parent: `<TodoItem onDelete={handleDelete} />`
- Child: `<button onClick={() => onDelete(todo.id)}>Delete</button>`
- Type the callback in the props interface: `onDelete: (id: number) => void`



Parent-Child Communication

```
// TodoItem.tsx
interface TodoItemProps {
  id: number;
  text: string;
  done: boolean;
  onToggle: (id: number) => void;
  onDelete: (id: number) => void;
}

function TodoItem({ id, text, done, onToggle, onDelete }: TodoItemProps) {
  return (
    <li>
      <span
        style={{ textDecoration: done ? 'line-through' : 'none' }}
        onClick={() => onToggle(id)}
      >
        {text}
      </span>
      <button onClick={() => onDelete(id)}>Delete</button>
    </li>
  );
}
```



React DevTools

What It Shows

- The full component tree — every component, its props, and its state
- Click any component to inspect its current props and state values

Getting Started

- Install the React Developer Tools browser extension (Chrome or Firefox)
- Open DevTools (F12) → Components tab
- Profiler tab for performance analysis (covered in a later course)

Debugging Tips

- Check props and state to verify data flows correctly
- Watch for unnecessary re-renders by highlighting updates



Key Takeaways

Key Takeaways

1. React builds UIs from reusable function components — each returns JSX describing what to render
2. Props are typed read-only arguments: define an interface, destructure in the parameter list
3. `useState` manages data that changes — always use the setter, never mutate directly
4. Events use camelCase (`onClick`, `onChange`) with typed handlers (`React.ChangeEvent<HTMLInputElement>`)
5. Lists require unique key props — use IDs from data, never array indices
6. Components communicate: data flows down via props, events flow up via callbacks

